



**Facultad de
Ciencias**

UNAM

PRÁCTICA 6 - MODELO FORK-JOIN

ALGORITMOS PARALELOS

Baltodano Cuevas Erick Joel

Profesor: Mario Arturo Nieto Butron

1. Objetivos

- Implementar una estructura de datos para representar partículas en un espacio tridimensional.
- Generar partículas con posiciones y masas aleatorias.
- Calcular la energía potencial gravitacional total entre partículas.
- Utilizar OpenMP para paralelizar el cálculo mediante el modelo Fork-Join.
- Analizar el desempeño del programa variando el número de hilos.

2. Metodología

La práctica fue desarrollada en lenguaje C utilizando la biblioteca OpenMP para implementar paralelismo en memoria compartida.

2.1. Definición del problema

El problema consiste en calcular la energía potencial gravitacional total entre un conjunto de partículas distribuidas aleatoriamente en el espacio tridimensional.

Cada partícula posee:

- Coordenadas (x, y, z)
- Masa

La interacción gravitacional depende de la distancia entre partículas y de sus masas.

2.2. Estructura de datos

Se definió la siguiente estructura para representar una partícula:

```
1 typedef struct {  
2     double x, y, z;  
3     double masa;  
4 } Particula;
```

2.3. Inicialización de partículas

Se creó un arreglo de $N = 1000$ partículas.

Para cada partícula se generaron:

- Posiciones aleatorias dentro del rango máximo definido.
- Masas aleatorias positivas.

La inicialización se realizó utilizando la función `rand()`.

2.4. Cálculo de distancia

La distancia euclidiana entre dos partículas se calculó mediante:

$$r_{ij} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2 + (z_i - z_j)^2}$$

Para evitar divisiones entre cero se utilizó un valor pequeño ϵ .

2.5. Cálculo de energía gravitacional

La energía potencial gravitacional total fue calculada con la expresión:

$$U = -G \sum_{i=1}^n \sum_{j=i+1}^n \frac{m_i m_j}{r_{ij}}$$

donde:

- G es la constante gravitacional.
- m_i y m_j representan las masas.
- r_{ij} representa la distancia entre partículas.

2.6. Paralelización con OpenMP

El cálculo fue paralelizado utilizando el modelo Fork-Join de OpenMP mediante la directiva:

```
1 #pragma omp parallel for
```

Se utilizaron las siguientes cláusulas:

- `num_threads(num_hilos)`
- `schedule(dynamic,16)`
- `reduction(+:U)`
- `default(none)`

La cláusula `reduction` permitió combinar correctamente las sumas parciales calculadas por cada hilo y evitar condiciones de carrera.

2.7. Ejecución del programa

El número de hilos se recibe desde la línea de comandos:

```
1 ./gravitacional <num_hilos>
```

Ejemplo:

```
1 ./gravitacional 4
```

3. Resultados

El programa muestra:

- El número total de partículas.
- El número de hilos utilizados.
- Una muestra de las primeras partículas generadas.
- La energía potencial gravitacional total.
- El tiempo de ejecución.

```
baltimus@Dev:~/github/Practicas_Paralelos/Practica6$ make run N=4
gcc -fopenmp -o src/Main src/Main.c -lm
./src/Main 4
=====
Práctica #6 – Energía Potencial Gravitacional
Modelo Fork-Join con OpenMP
=====

Partículas : 1000
Hilos      : 4

Muestra de las primeras 5 partículas:
ID    x (m)      y (m)      z (m)      masa (kg)
-----
0     3.700e+10   6.419e+11   2.344e+11   7.403e+29
1     3.269e+09   4.630e+11   6.629e+11   3.567e+29
2     2.233e+11   4.347e+11   2.073e+11   1.542e+29
3     4.225e+11   6.982e+11   5.402e+11   4.637e+29
4     2.922e+11   8.888e+11   9.305e+11   1.080e+29

--- Cálculo PARALELO (Fork-Join con OpenMP) ---
U total = -1.578182e+43 J
Tiempo  = 0.0115 s

rm -r src/Main
```

Se observó que el tiempo de ejecución disminuye al aumentar el número de hilos, debido a que el trabajo es distribuido entre varios procesadores.

El uso de OpenMP permitió paralelizar el cálculo sin modificar significativamente la estructura principal del programa.

También se observó que la distribución dinámica del trabajo mediante:

```
1 schedule(dynamic,16)
```

mejora el balance de carga entre los hilos.

4. Conclusiones

- Se implementó correctamente una simulación de energía potencial gravitacional utilizando OpenMP.
- El modelo Fork-Join permitió distribuir el cálculo entre múltiples hilos de ejecución.

- La directiva `parallel for` simplificó el proceso de paralelización.
- La cláusula `reduction` fue necesaria para evitar errores por acceso concurrente a la variable acumuladora.
- El tiempo de ejecución disminuye conforme aumenta el número de hilos disponibles.
- OpenMP representa una herramienta eficiente y sencilla para desarrollar aplicaciones paralelas en sistemas de memoria compartida.